In-House Authentication Plan — Next.js App Router (Revised)

This supersedes the original draft. Every item below was identified as a gap or bug across four rounds of review and has a concrete fix baked in. Sections marked ⚠ **Known limitation** are deliberate, documented trade-offs rather than oversights.

---

1. Decisions locked in

Decision	Choice	Why
Database	**PostgreSQL** everywhere (Docker locally, Neon/Supabase in prod)	SQLite files don't persist on serverless platforms (Vercel/Lambda ephemeral filesystem). No dev/prod parity risk.
Refresh token format	**Opaque random token**, hashed (SHA-256) before storage	JWTs can't be revoked before expiry; opaque + DB lookup can.
Refresh token revocation	**Family-based rotation with reuse detection**	Lets you kill exactly the compromised session chain without logging out every device on that user.
CSRF strategy	**Origin/Host header matching**, deny-by-default	`SameSite=Lax` already blocks the common case; Origin check is cheap defense-in-depth without double-submit-cookie complexity.
Edge Middleware scope	**Signature + expiry check only, no DB**	Prisma isn't Edge-runtime compatible without extra infra (Accelerate/Neon HTTP driver). Keep Edge fast; push real revocation checks to the Node-runtime refresh route.
Account states	`ACTIVE`, `SUSPENDED`, `PENDING_VERIFICATION`	Verification flow is now actually wired to this state (see §6).

---

2. Database Schema (`prisma/schema.prisma`)

```
prisma
```

```
enum Role {
  ADMIN
  MANAGER
  LANDLORD
  TENANT
  VENDOR
}

enum UserStatus {
  ACTIVE
  SUSPENDED
  PENDING_VERIFICATION
}

model User {
  id                String             @id @default(uuid())
  email             String             @unique
  name              String
  passwordHash      String
  role              Role               @default(TENANT)
  status            UserStatus         @default(PENDING_VERIFICATION)
  lastLoginAt       DateTime?
  refreshTokens     RefreshToken[]
  verificationToken VerificationToken?
  loginAttempts     LoginAttempt[]
  createdAt         DateTime           @default(now())
  updatedAt         DateTime           @updatedAt
}

model RefreshToken {
  id         String    @id @default(uuid())
  userId     String
  tokenHash  String    @unique // sha256 of the raw opaque token — never store
  familyId   String    // groups all tokens descended from one login
  revokedAt  DateTime?
  replacedBy String?   // tokenHash of the child that replaced this one
  expiresAt  DateTime
  createdAt  DateTime  @default(now())
  user       User      @relation(fields: [userId], references: [id], onDelete:

  @@index([familyId])
}
```

```
model VerificationToken {
  id        String   @id @default(uuid())
  userId    String   @unique
  tokenHash String   @unique
  expiresAt DateTime
  createdAt DateTime @default(now())
  user      User     @relation(fields: [userId], references: [id], onDelete: Ca
}

// Fallback rate limiting if Upstash/Redis isn't set up yet.
// Prefer @upstash/ratelimit in production — this table works with zero extra i
model LoginAttempt {
  id         String   @id @default(uuid())
  identifier String   // ip address, or ip+email combo
  userId     String?
  user       User?    @relation(fields: [userId], references: [id])
  createdAt  DateTime @default(now())

  @@index([identifier, createdAt])
}
```

---

3. Core Auth Utilities (`lib/auth/`)

`lib/auth/jwt.ts`

typescript

```typescript
import { SignJWT, jwtVerify } from 'jose';

const JWT_SECRET = new TextEncoder().encode(process.env.JWT_SECRET!);

export interface JWTPayload {
  sub: string;
  role: string;
}

export async function signAccessToken(payload: JWTPayload): Promise<string> {
  return new SignJWT({ ...payload })
    .setProtectedHeader({ alg: 'HS256' })
    .setIssuedAt()
    .setExpirationTime('15m')
    .sign(JWT_SECRET);
}

export async function verifyToken(token: string): Promise<JWTPayload | null> {
  try {
    const { payload } = await jwtVerify(token, JWT_SECRET);
    return payload as unknown as JWTPayload;
  } catch {
    return null;
  }
}
```

`lib/auth/cookies.ts`

Cookie deletion must match `path` exactly — a mismatched path silently no-ops and leaves the stale cookie in the browser.

typescript

```typescript
import { cookies } from 'next/headers';

export async function setAuthCookies(accessToken: string, refreshToken: string)
  const cookieStore = await cookies();

  cookieStore.set('access_token', accessToken, {
    httpOnly: true,
    secure: process.env.NODE_ENV === 'production',
    sameSite: 'lax',
    path: '/',
    maxAge: 15 * 60,
  });

  cookieStore.set('refresh_token', refreshToken, {
    httpOnly: true,
    secure: process.env.NODE_ENV === 'production',
    sameSite: 'lax',
    path: '/api/auth/refresh', // never attached outside this endpoint
    maxAge: 7 * 24 * 60 * 60,
  });
}

export async function clearAuthCookies() {
  const cookieStore = await cookies();

  cookieStore.delete({ name: 'access_token', path: '/' });
  cookieStore.delete({ name: 'refresh_token', path: '/api/auth/refresh' }); //
}
```

`lib/auth/csrf.ts`

Deny-by-default when `Origin` is absent; fall back to `Referer` rather than allowing the request through.

typescript

```
import { NextRequest } from 'next/server';

export function validateCSRF(req: NextRequest): boolean {
  const host = req.headers.get('x-forwarded-host') || req.headers.get('host');
  if (!host) return false;

  const origin = req.headers.get('origin');
  if (origin) {
    return new URL(origin).host === host;
  }

  // No Origin header (some same-site non-fetch requests omit it) — fall back t
  const referer = req.headers.get('referer');
  if (referer) {
    try {
      return new URL(referer).host === host;
    } catch {
      return false;
    }
  }

  // Neither header present on a mutating request — reject rather than assume s
  return false;
}
```

`lib/auth/session.ts` — Server Component session access

Read the cookie directly in Server Components instead of relying on client-side `AuthContext`, to avoid a client fetch waterfall and layout shift on first paint.

typescript

```
import { cookies } from 'next/headers';
import { verifyToken, JWTPayload } from '@/lib/auth/jwt';

export async function getServerSession(): Promise<JWTPayload | null> {
  const cookieStore = await cookies();
  const token = cookieStore.get('access_token')?.value;
  if (!token) return null;
  return verifyToken(token);
}
```

`lib/auth/rateLimit.ts` — simple DB-backed limiter (swap for Upstash later)

typescript

```typescript
import { prisma } from '@/lib/db';

const WINDOW_MS = 5 * 60 * 1000; // 5 minutes
const MAX_ATTEMPTS = 5;

export async function checkRateLimit(identifier: string): Promise<boolean> {
  const since = new Date(Date.now() - WINDOW_MS);
  const count = await prisma.loginAttempt.count({
    where: { identifier, createdAt: { gt: since } },
  });
  return count < MAX_ATTEMPTS;
}

export async function recordAttempt(identifier: string, userId?: string) {
  await prisma.loginAttempt.create({ data: { identifier, userId } });
}

// x-forwarded-for is a comma-separated proxy chain ("client, proxy1, proxy2")
// Vercel/ALB/Cloudflare — always take the first entry, and note that this head
// is client-suppliable unless your platform strips it before your app sees it,
// so don't treat it as authoritative behind an untrusted or misconfigured prox
export function getClientIp(req: { headers: { get(name: string): string | null
  const raw = req.headers.get('x-forwarded-for') ?? '127.0.0.1';
  return raw.split(',')[0].trim();
}
```

---

4. API Route Handlers (`app/api/auth/`)

`POST /api/auth/refresh` — atomic rotation with reuse detection

The rotation check must be a single atomic UPDATE, not a read-then-write — otherwise two concurrent refresh calls carrying the same token can both pass validation before either write lands, silently double-issuing sessions.

typescript

```ts
import { NextRequest, NextResponse } from 'next/server';
import crypto from 'crypto';
import { prisma } from '@/lib/db';
import { signAccessToken } from '@/lib/auth/jwt';
import { setAuthCookies, clearAuthCookies } from '@/lib/auth/cookies';
import { validateCSRF } from '@/lib/auth/csrf';
import { checkRateLimit, getClientIp } from '@/lib/auth/rateLimit';

export async function POST(req: NextRequest) {
  if (!validateCSRF(req)) {
    return NextResponse.json({ error: 'Cross-origin request blocked' }, { statu
  }

  // Not credential-guessing risk (the token is 256 bits of entropy — unguessab
  // regardless of request volume); this is DoS/resource-exhaustion protection,
  // since every call here hits the database at least once.
  if (!(await checkRateLimit(`refresh:${getClientIp(req)}`))) {
    return NextResponse.json({ error: 'Too many requests' }, { status: 429 });
  }

  const rawRefreshToken = req.cookies.get('refresh_token')?.value;
  if (!rawRefreshToken) {
    return NextResponse.json({ error: 'Missing refresh token' }, { status: 401
  }

  const tokenHash = crypto.createHash('sha256').update(rawRefreshToken).digest(

  // Atomic: only succeeds if this token is currently valid and unrevoked.
  const rotated = await prisma.refreshToken.updateMany({
    where: { tokenHash, revokedAt: null, expiresAt: { gt: new Date() } },
    data: { revokedAt: new Date() },
  });

  if (rotated.count === 0) {
    // Either invalid/expired, or already revoked (= reuse of a rotated-out tok
    const existing = await prisma.refreshToken.findUnique({ where: { tokenHash
    if (existing?.revokedAt) {
      await prisma.refreshToken.updateMany({
        where: { familyId: existing.familyId },
        data: { revokedAt: new Date() },
      });
    }
    await clearAuthCookies();
```

```
    return NextResponse.json({ error: 'Invalid or revoked session' }, { status:
  }

  const currentToken = await prisma.refreshToken.findUnique({
    where: { tokenHash },
    include: { user: true },
  });
  if (!currentToken || currentToken.user.status !== 'ACTIVE') {
    await clearAuthCookies();
    return NextResponse.json({ error: 'Account inactive' }, { status: 403 });
  }

  const newRawRefreshToken = crypto.randomBytes(32).toString('hex');
  const newTokenHash = crypto.createHash('sha256').update(newRawRefreshToken).d

  // Wrapped in a transaction for crash-safety: if the process dies between the
  // writes, we don't want a parent left pointing at a child that was never cre
  // (The atomic updateMany above already rules out two callers racing to reach
  // point for the same tokenHash — this transaction is about write atomicity,
  await prisma.$transaction([
    prisma.refreshToken.update({
      where: { id: currentToken.id },
      data: { replacedBy: newTokenHash },
    }),
    prisma.refreshToken.create({
      data: {
        userId: currentToken.userId,
        tokenHash: newTokenHash,
        familyId: currentToken.familyId,
        expiresAt: new Date(Date.now() + 7 * 24 * 60 * 60 * 1000),
      },
    }),
  ]);

  const newAccessToken = await signAccessToken({
    sub: currentToken.user.id,
    role: currentToken.user.role,
  });
  await setAuthCookies(newAccessToken, newRawRefreshToken);

  return NextResponse.json({ success: true });
}
```

`POST /api/auth/login` — with rate limiting and status checks

typescript

```ts
import { NextRequest, NextResponse } from 'next/server';
import crypto from 'crypto';
import bcrypt from 'bcryptjs';
import { prisma } from '@/lib/db';
import { signAccessToken } from '@/lib/auth/jwt';
import { setAuthCookies } from '@/lib/auth/cookies';
import { checkRateLimit, recordAttempt, getClientIp } from '@/lib/auth/rateLimi
import { validateCSRF } from '@/lib/auth/csrf';

export async function POST(req: NextRequest) {
  if (!validateCSRF(req)) {
    return NextResponse.json({ error: 'Cross-origin request blocked' }, { statu
  }

  const { email, password } = await req.json();
  const identifier = `${getClientIp(req)}:${email}`;

  if (!(await checkRateLimit(identifier))) {
    return NextResponse.json({ error: 'Too many attempts. Try again later.' },
  }

  const user = await prisma.user.findUnique({ where: { email } });
  const valid = user && (await bcrypt.compare(password, user.passwordHash));

  if (!valid) {
    await recordAttempt(identifier, user?.id);
    return NextResponse.json({ error: 'Invalid credentials' }, { status: 401 })
  }

  if (user.status === 'SUSPENDED') {
    return NextResponse.json({ error: 'Account suspended' }, { status: 403 });
  }
  if (user.status === 'PENDING_VERIFICATION') {
    return NextResponse.json({ error: 'Please verify your email first' }, { sta
  }

  // non-blocking audit write
  prisma.user.update({ where: { id: user.id }, data: { lastLoginAt: new Date()

  const familyId = crypto.randomUUID();
  const rawRefreshToken = crypto.randomBytes(32).toString('hex');
  const tokenHash = crypto.createHash('sha256').update(rawRefreshToken).digest(
```

```
  await prisma.refreshToken.create({
    data: {
      userId: user.id,
      tokenHash,
      familyId,
      expiresAt: new Date(Date.now() + 7 * 24 * 60 * 60 * 1000),
    },
  });

  const accessToken = await signAccessToken({ sub: user.id, role: user.role });
  await setAuthCookies(accessToken, rawRefreshToken);

  return NextResponse.json({ success: true, user: { id: user.id, role: user.rol
}
```

`POST /api/auth/change-password` — global session invalidation

typescript

```
import { NextRequest, NextResponse } from 'next/server';
import bcrypt from 'bcryptjs';
import { prisma } from '@/lib/db';
import { getServerSession } from '@/lib/auth/session';
import { clearAuthCookies } from '@/lib/auth/cookies';
import { validateCSRF } from '@/lib/auth/csrf';

export async function POST(req: NextRequest) {
  if (!validateCSRF(req)) {
    return NextResponse.json({ error: 'Cross-origin request blocked' }, { statu
  }

  const session = await getServerSession();
  if (!session) return NextResponse.json({ error: 'Unauthorized' }, { status: 4

  const { currentPassword, newPassword } = await req.json();
  const user = await prisma.user.findUnique({ where: { id: session.sub } });

  if (!user || !(await bcrypt.compare(currentPassword, user.passwordHash))) {
    return NextResponse.json({ error: 'Invalid current password' }, { status: 4
  }

  const newPasswordHash = await bcrypt.hash(newPassword, 12);

  await prisma.$transaction([
    prisma.user.update({ where: { id: user.id }, data: { passwordHash: newPassw
    prisma.refreshToken.updateMany({
      where: { userId: user.id, revokedAt: null },
      data: { revokedAt: new Date() },
    }),
  ]);

  await clearAuthCookies();
  return NextResponse.json({ success: true, message: 'Password updated. Please
}
```

`POST /api/auth/verify-email` — completes the `PENDING_VERIFICATION` flow

typescript

```
import { NextRequest, NextResponse } from 'next/server';
import crypto from 'crypto';
import { prisma } from '@/lib/db';

// Deliberately NOT wrapped in validateCSRF: this route is meant to be reached
// clicking a link from an email client, which is a cross-origin navigation by
// design. The single-use, time-limited verification token is the actual securi
// boundary here, not Origin matching — adding a CSRF check would just break
// legitimate verification links. Do not "fix" this by adding validateCSRF().
export async function POST(req: NextRequest) {
  const { token } = await req.json();
  const tokenHash = crypto.createHash('sha256').update(token).digest('hex');

  const record = await prisma.verificationToken.findUnique({ where: { tokenHash
  if (!record || record.expiresAt < new Date()) {
    return NextResponse.json({ error: 'Invalid or expired token' }, { status: 4
  }

  await prisma.$transaction([
    prisma.user.update({ where: { id: record.userId }, data: { status: 'ACTIVE'
    prisma.verificationToken.delete({ where: { id: record.id } }),
  ]);

  return NextResponse.json({ success: true });
}
```

`register` should create a `VerificationToken` and email it (Resend or similar) instead of setting the user `ACTIVE` immediately — otherwise this table and status are unused.

---

5. Middleware (`middleware.ts`)

Deliberately dumb and fast: signature + expiry only. No DB call — Prisma isn't Edge-compatible without extra infra (Accelerate/Neon HTTP driver), and adding one here would slow every request.

typescript

```ts
import { NextResponse } from 'next/server';
import type { NextRequest } from 'next/server';
import { verifyToken } from '@/lib/auth/jwt';

const PROTECTED_ROUTES = ['/dashboard', '/admin'];

export async function middleware(req: NextRequest) {
  const { pathname } = req.nextUrl;
  if (!PROTECTED_ROUTES.some((route) => pathname.startsWith(route))) {
    return NextResponse.next();
  }

  const token = req.cookies.get('access_token')?.value;
  const payload = token ? await verifyToken(token) : null;

  if (!payload) {
    const loginUrl = new URL('/login', req.url);
    loginUrl.searchParams.set('from', pathname);
    return NextResponse.redirect(loginUrl);
  }

  if (pathname.startsWith('/admin') && payload.role !== 'ADMIN') {
    return NextResponse.redirect(new URL('/dashboard', req.url));
  }

  return NextResponse.next();
}

export const config = { matcher: ['/dashboard/:path*', '/admin/:path*'] };
```

> ⚠ **Known limitation:** middleware only checks token validity, not live account status. A suspended user's existing `access_token` still passes middleware until it expires (max 15 min) or until they hit `/api/auth/refresh`, which does check status. If instant kill is required, add a Redis-backed blocklist keyed by `userId`/`familyId` and check it here — this is optional infrastructure, not required for v1.

---

6. Client Integration (`lib/apiClient.ts` + `AuthContext`)

Deduplicates concurrent refresh attempts so multiple parallel API calls hitting an expired token don't race each other into a false "reuse detected" logout.

typescript

```
let refreshPromise: Promise<boolean> | null = null;

async function refreshAccessToken(): Promise<boolean> {
  if (!refreshPromise) {
    refreshPromise = fetch('/api/auth/refresh', { method: 'POST' })
      .then((res) => res.ok)
      .finally(() => {
        refreshPromise = null;
      });
  }
  return refreshPromise;
}

export async function apiFetch(url: string, options: RequestInit = {}) {
  let res = await fetch(url, options);

  if (res.status === 401) {
    const refreshed = await refreshAccessToken();
    if (refreshed) {
      res = await fetch(url, options);
    } else {
      window.location.href = '/login';
    }
  }

  return res;
}
```

Resolved: navigation bouncing is handled with a proactive silent-refresh timer, kept *alongside* `apiFetch` (not instead of it) — background-tab timer throttling means the reactive interceptor is still needed as a backstop for whatever the timer misses.

`hooks/useAuthRefresh.ts` — proactive timer, multi-tab safe

Cookies are shared across all tabs on the same origin. If two tabs' timers fire close together, only one refresh call can win the atomic rotation (§4) — the other gets a 401 for a session that is, from the user's point of view, still perfectly valid. **Never hard-redirect on a failed silent refresh without first re-checking whether the session is actually dead**, or a losing tab will log the user out from under them while the winning tab stays signed in.

typescript

```ts
'use client';

import { useEffect } from 'react';

export function useAuthRefresh() {
  useEffect(() => {
    // Fire ~2 minutes before the 15-minute access token expires.
    const INTERVAL_MS = 13 * 60 * 1000;

    const performSilentRefresh = async () => {
      try {
        const res = await fetch('/api/auth/refresh', { method: 'POST' });
        if (res.ok) return;

        // This tab's refresh attempt failed — but another tab may have already
        // the rotation race and left a valid session in the shared cookie jar.
        // Confirm the session is truly dead before redirecting.
        const check = await fetch('/api/auth/me');
        if (!check.ok) {
          window.location.href = '/login';
        }
      } catch {
        // Network hiccup — let the next interval tick retry rather than loggin
      }
    };

    const timer = setInterval(performSilentRefresh, INTERVAL_MS);
    return () => clearInterval(timer);
  }, []);
}
```

Mount `useAuthRefresh()` once near the root of the authenticated layout (e.g. inside `AuthContext`'s provider), not per-page, so it doesn't spawn multiple intervals.

`AuthContext` (`context/AuthContext.tsx`) wraps `user`, `loading`, `login()`, `register()`, `logout()`, calls `useAuthRefresh()` on mount, and should route protected `/api/*` calls through `apiFetch` (not raw `fetch`) so the reactive interceptor still catches whatever the timer misses (e.g. a backgrounded tab where the browser throttled `setInterval`).

---

7. Verification Plan

Automated

- `pnpm dev` + endpoint tests via `curl`/API tests.
- Edge cases: invalid password, suspended/pending status, expired access token → refresh, role redirection, **concurrent refresh calls** (fire 3+ parallel requests with the same refresh token and confirm exactly one succeeds, others get a clean 401 — not a false reuse-detection wipe).

Manual

1. **Cookie inspection** — confirm `access_token` (`path=/`) and `refresh_token` (`path=/api/auth/refresh`) both show `HttpOnly`, `SameSite=Lax`, and (in prod) `Secure`.
2. **Suspension test** — set `status = 'SUSPENDED'` mid-session; confirm access continues until token expiry, then `/api/auth/refresh` returns 403.
3. **Reuse detection test** — capture a refresh token, use it once (rotates successfully), then replay the *original* raw token; confirm the entire family is revoked and the legitimate session is also logged out.
4. **RBAC test** — non-admin hitting `/admin` redirects to `/dashboard`.
5. **Logout test** — confirm both cookies are actually removed from the browser (not just server-side revoked) by checking DevTools → Application → Cookies after logout.
6. **CSRF test** — POST to `/api/auth/change-password` from a different origin (e.g. via a local test HTML page) and confirm 403.
7. **Rate limit test** — 6 failed logins in under 5 minutes from the same identifier returns 429 on the 6th.

---

8. Decisions

- **Instant revocation (Redis blocklist): deferred for v1.** The 15-min `access_token` TTL bounds worst-case exposure for a suspended/compromised account; `/api/auth/refresh` catches it fully within that window. Revisit only if a product requirement for instant kill emerges — don't add the operational cost preemptively.
- **Navigation bouncing: resolved with the multi-tab-safe silent refresh timer in §6**, running alongside (not replacing) the reactive `apiFetch` interceptor.
- **Rate limiting: prefer `@upstash/ratelimit` at scale**; the DB-backed `LoginAttempt` table in §2/§3 stays in the schema as a zero-dependency fallback for local dev/testing before Upstash is wired in — no need to rip it out once Upstash is added.
- **Email provider for verification tokens** — still open, pick one (Resend, Postmark, etc.) before wiring `POST /api/auth/register` to actually send the email.

Everything else in this document is resolved and ready to implement in the order: schema → cookie/JWT/CSRF utils → login/refresh routes (with `$transaction`-wrapped rotation and CSRF check) → middleware → client interceptor + silent refresh timer → verification + password-change routes.